

第 7 週：時序模擬

本週一句話：前 6 週的模擬都是「假設閘是瞬間反應」。真實電路不是。

本週指令

雙擊 `env.bat` 開好環境（提示字元要有 `[OSS CAD Suite]`），然後：

```
cd week07_timing_sim
```

每個範例都是這三條（外加一條檢查），把 `02_glitch` 換成你要跑的名字：

```
iverilog -o sim.out 02_glitch.v 02_glitch_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 02_glitch.gtkw
```

`echo %ERRORLEVEL%` 印 0 才是編譯成功——失敗時 iverilog 常常一個字都不印。

本週範例名稱：

- `01_gate_delay`
- `02_glitch`
- `03_cdc_sync`
- `04_netlist`

下面每個範例的段落，都直接附了它自己的四行指令。

學完你要會什麼

- [] 知道功能模擬和時序模擬差在哪

- [] 懂什麼是傳播延遲、關鍵路徑
- [] ★ 知道毛刺 (glitch) 怎麼產生的，以及為什麼平常不會出事
- [] 知道亞穩態，會寫雙 FF 同步器
- [] ★★ 跑過閘級模擬，親眼看到真實電路的樣子

一、兩種模擬

	功能模擬 (前 6 週)	時序模擬 (本週)
別名	RTL 模擬、行為模擬	閘級模擬、post-layout 模擬
模擬什麼	你寫的 Verilog	綜合 / 佈局後的真實網表
延遲	零	每顆閘、每條線都有
速度	快	慢很多
看得到毛刺嗎	✗	✓
什麼時候做	天天做	設計快定案時

二、傳播延遲與關鍵路徑

```
assign #2 n1 = a & b;      // AND 閘花 2ns
assign #2 y  = n1 | c;     // OR 閘再花 2ns → 總共 4ns
```

關鍵路徑 = 整個電路裡**最長**的那條組合邏輯路徑。

最高時脈 = 1 / 關鍵路徑延遲

例：關鍵路徑 4ns → 最高 250 MHz

這是硬體設計最重要的效能指標。第 12 週的管線化，就是把**長路徑切短**來拉高時脈。

三、★ 毛刺 glitch

```
y = a | ~a          // 數學上永遠是 1
```

但 `~a` 那條路要多穿過一顆反相器。 `a` 從 1 變 0 的瞬間：

```
a 已經變 0
~a 還在路上 (還是 0)
→ 這幾 ns 之內 y = 0 | 0 = 0 ← 毛刺！
```

實測 (範例 02)：抓到 2 次毛刺。

[!IMPORTANT] 為什麼平常不會出事？因為輸出都接到正反器，正反器只在時脈邊緣取樣。只要毛刺在下一個邊緣之前消失，就完全沒影響。

什麼時候會出事？

1. 用組合邏輯直接當時脈 (絕對禁止)
2. 用組合邏輯直接當非同步 reset
3. 訊號直接接出晶片外

這就是「同步設計」原則的由來：所有東西都用同一個時脈邊緣取樣。

四、亞穩態與雙 FF 同步器

外來訊號 (按鈕、另一個時脈域的資料) 如果剛好在時脈邊緣附近變化，正反器會進入亞穩態——輸出卡在 0 和 1 之間，要花不定的時間才穩定，還可能傳染給下游造成整個系統當掉。

解法：雙 FF 同步器

```
always @(posedge clk) begin
    sync_ff1 <= async_in;          // 第一顆可能亞穩態
    safe_out <= sync_ff1;          // 給它一整個週期恢復，第二顆取到乾淨的值
end
```

代價：延遲兩拍。好處：不會當機。

[!WARNING] 模擬看不出亞穩態 —— 那是類比現象。這是少數必須靠「知道規則」而不是靠模擬來避免的錯誤。

規則：任何跨時脈域的訊號，一律先過雙 FF 同步器。

五、本週四個範例

```
cd week07_timing_sim
```

📁 01_gate_delay — 閘延遲

```
iverilog -o sim.out 01_gate_delay.v 01_gate_delay_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 01_gate_delay.gtkw
```

理想版和真實版並排。波形上 `y_real` 永遠比 `y_ideal` 晚 4ns。

📁 02_glitch — 毛刺 ★

```
iverilog -o sim.out 02_glitch.v 02_glitch_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 02_glitch.gtkw
```

`y = a | ~a` 這個「永遠是 1」的電路，實際上會閃 0。

📁 03_cdc_sync — 跨時脈域同步器

```
iverilog -o sim.out 03_cdc_sync.v 03_cdc_sync_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 03_cdc_sync.gtkw
```

波形上看三條線的階梯關係：`async_in` → `sync_ff1` → `safe_out`。

04_netlist — ★★ 閘級模擬 (本週重頭戲)

```
iverilog -gspecify -DICE40_HX -DNO_ICE40_DEFAULT_ASSIGNMENTS -o sim.out
04_netlist.v 04_netlist_tb.v
echo %ERRORLEVEL%
vvp sim.out
gtkwave 04_netlist.gtkw
```

這個範例做的事：

1. `synth.bat` 用 **Yosys** 把設計綜合成真實的 iCE40 網表 (`04_netlist_syn.v`)
2. 測試平台同時跑你的 RTL 和那份閘級網表
3. 搭配 `-gspecify -DICE40_HX`，iverilog 會套用 **Lattice 原廠的真實延遲數字**

實測：

```
a= 0 b= 0 | RTL: sum= 0 big=0 | 閘級: sum= 0 big=0 一致
a=255 b=255 | RTL: sum=510 big=1 | 閘級: sum=510 big=1 一致
>>> 穩定之後兩邊完全一致 (功能正確)
>>> 但閘級版的 big 總共跳動了 4 次
```

功能一樣，過程完全不同。

[!IMPORTANT] ★ 一定要做的波形操作：

1. 把 `sum_rtl` 和 `sum_gate` 上下排在一起
2. 找 `a` 從 0 跳到 255 的那個時間點
3. 一直按 **Zoom In** 放大到 ns 等級
4. 你會看到 `sum_gate` 在幾 ns 之內連跳好幾個中途錯值才穩定 而 `sum_rtl` 是一步到位

那些中途錯值就是進位一級一級傳過 8 個 `SB_CARRY` 的過程。這是你到目前為止離「真實硬體」最近的一次。

想重新綜合：在這個資料夾執行 `synth.bat`。

六、練習題

練習 1

把 `01_gate_delay.v` 的延遲改成 `#5`，重跑，確認波形上的偏移變成 10ns。

練習 2

在 `02_glitch.v` 把 `y_real` 接到一個正反器，用 10ns 的時脈取樣，確認取樣後的輸出沒有毛刺（驗證同步設計原則）。

練習 3 ★

改 `_synth_src.v` 的設計（例如加大到 16 位元），跑 `synth.bat` 重新綜合，再跑 `sim 04_netlist`，看毛刺變多還是變少、穩定時間變長還是變短。

練習 4 (思考)

為什麼下面這樣寫是大忌？

```
wire slow_clk = counter[10];           // 用計數器的某一位當時脈
always @(posedge slow_clk) ...
```

► 看答案

七、本週檢核

- ☐ 四個範例都跑過
 - ☐ `02` 的波形上指得出毛刺那一小段
 - ☐ `04` 有放大到 ns 等級，看到 `sum_gate` 的中途錯值
 - ☐ 說得出關鍵路徑跟最高時脈的關係
 - ☐ 練習 4 三個理由至少答出兩個
-

附註：關於 `sim` 這個指令

你可能在別的地方看過我寫的 `sim` 範例名稱。那是我自己做的批次檔（`sim.bat`），不是 Verilog 或 iverilog 的標準指令，教科書上查不到。

它做的事就是把上面那四行包起來、順便幫你檢查結束碼。用不用都可以，也可以刪掉。這份 README 裡給的全部都是原始指令。